

Python Notes — Nested Dictionaries

1. What is a Nested Dictionary?

A **nested dictionary** is a dictionary that contains another dictionary as its value.

Example:

```
employees = {  
    "emp1": {  
        "name": "Rohit",  
        "department": "IT",  
        "salary": 65000  
    },  
    "emp2": {  
        "name": "Amit",  
        "department": "HR",  
        "salary": 50000  
    }  
}
```

Here:

```
employees  
  ↓  
emp1  
  ↓  
another dictionary  
  ├── name  
  ├── department  
  └── salary
```

2. Accessing Nested Dictionary Values

To access a value inside a nested dictionary, use multiple keys.

Access the complete employee record

```
employees["emp1"]
```

Output:

```
{
  "name": "Rohit",
  "department": "IT",
  "salary": 65000
}
```

Access a specific value

```
employees["emp1"]["name"]
```

Output:

```
Rohit
```

```
employees["emp1"]["salary"]
```

Output:

```
65000
```

General pattern

```
dictionary["outer_key"]["inner_key"]
```

3. Adding a New Key Inside a Nested Dictionary

You can add a new key directly:

```
employees["emp1"]["city"] = "Mumbai"
```

Now:

```
"emp1": {
  "name": "Rohit",
```

```
"department": "IT",  
"salary": 65000,  
"city": "Mumbai"  
}
```

If the key already exists, this syntax updates its value.

4. Updating a Nested Value

Example:

```
employees["emp1"]["salary"] = 70000
```

Before:

```
salary → 65000
```

After:

```
salary → 70000
```

General pattern

```
dictionary["outer_key"]["inner_key"] = new_value
```

5. Adding a New Nested Dictionary

To add a completely new employee:

```
employees["emp4"] = {  
    "name": "Neha",  
    "department": "Marketing",  
    "salary": 60000,  
    "city": "Mumbai"  
}
```

Here:

```
employees
  ↓
emp4 → new dictionary
    ↓
    name
    department
    salary
    city
```

Important

There are two levels:

```
employees["emp4"]
```

Accesses the **outer dictionary key**.

While:

```
employees["emp4"]["name"]
```

Accesses a value inside the **inner dictionary**.

6. Deleting a Nested Key

Suppose:

```
"emp4": {
  "name": "Neha",
  "department": "Marketing",
  "salary": 60000,
  "city": "Mumbai"
}
```

To remove only `"city"`:

```
del employees["emp4"]["city"]
```

Or:

```
employees["emp4"].pop("city")
```

The employee remains; only the `"city"` key is removed.

7. `pop()` With Nested Dictionaries

`pop()` removes a key and **returns its value**.

```
result = employees["emp4"].pop("city")
```

Now:

```
print(result)
```

gives:

```
Mumbai
```

And `"city"` is removed from `emp4`.

`pop()` **with a default**

```
result = employees["emp4"].pop("phone", "Not Available")
```

If `"phone"` doesn't exist:

```
Not Available
```

is returned instead of raising `KeyError`.

8. Deleting an Entire Nested Record

To delete the complete employee:

```
del employees["emp3"]
```

This removes:

```
emp3
├── name
├── department
└── salary
```

completely.

Another option:

```
employees.pop("emp3")
```

Important Difference

```
employees["emp3"].clear()
```

✗ Does NOT delete `emp3`.

It produces:

```
"emp3": {}
```

Whereas:

```
del employees["emp3"]
```

✓ Deletes `emp3` completely.

9. Nested Dictionary `clear()`

There are two important levels of `clear()`.

Clear only one employee

```
employees["emp1"].clear()
```

Result:

```
"emp1": {}
```

Other employees remain unchanged.

Clear the entire dictionary

```
employees.clear()
```

Result:

```
{}
```

All employees are removed.

Remember

```
employees["emp1"].clear()
```

↓

clears only emp1

```
employees.clear()
```

↓

clears the entire employees dictionary

10. Looping Through a Nested Dictionary

Consider:

```
employees = {  
    "emp1": {  
        "name": "Rohit",  
        "department": "IT",  
        "salary": 65000  
    },  
    "emp2": {  
        "name": "Amit",  
        "department": "HR",  
        "salary": 50000  
    }  
}
```

Loop through employee IDs

```
for employee in employees:  
    print(employee)
```

Output:

```
emp1  
emp2
```

11. Using `.items()` With Nested Dictionaries

```
for employee, details in employees.items():  
    print(employee)  
    print(details)
```

Here:

```
employee → "emp1"  
details  → {  
    "name": "Rohit",  
    "department": "IT",  
    "salary": 65000  
}
```

Very Important

`details` is already a **dictionary**.

Therefore:

```
details
```

is correct.

But:

```
[details]
```

creates a **list containing the dictionary**.

Example:

```
high_salary_employees[employee] = details
```

✓ Correct

```
high_salary_employees[employee] = [details]
```

✗ Creates the wrong structure if you want a nested dictionary.

12. Nested Looping

If you want to print every key and value inside every employee:

```
for employee, details in employees.items():  
    print(employee)  
  
    for key, value in details.items():  
        print(key, value)
```

The outer loop gets the employee.

The inner loop gets the data inside that employee.

Structure:

```
Outer loop
  ↓
emp1
  ↓
Inner loop
  ├── name → Rohit
  ├── department → IT
  └── salary → 65000
```

13. Accessing Known Nested Keys

If you already know the inner keys you need, you don't need another loop.

Example:

```
for employee, details in employees.items():
    name = details["name"]
    salary = details["salary"]

    print(f"{name} → {salary}")
```

This is cleaner when you only need specific fields.

14. Filtering Nested Dictionaries

Example: Find employees whose salary is at least 55000.

```
for employee, details in employees.items():
    if details["salary"] >= 55000:
        print(details["name"], details["salary"])
```

Output:

```
Rohit 65000
Rahul 55000
```

15. Creating a New Filtered Nested Dictionary

You can create a new dictionary containing only selected employees.

```
high_salary_employees = {}

for employee, details in employees.items():
    if details["salary"] >= 55000:
        high_salary_employees[employee] = details
```

The important line is:

```
high_salary_employees[employee] = details
```

It preserves the complete nested employee record.

16. Multiple Conditions

You can combine conditions using `and`.

Example:

```
if details["department"] == "IT" and details["salary"] >= 60000:
```

This means **both conditions must be true**.

Example:

```
high_paid_it_employees = {}

for employee, details in employees.items():
    if details["department"] == "IT" and details["salary"] >= 60000:
        high_paid_it_employees[employee] = details
```

17. Counting From a Nested Dictionary

Example: Count employees in the IT department.

```
count = 0

for employee, details in employees.items():
    if details["department"] == "IT":
        count += 1

print(count)
```

General pattern

```
count = 0

for key, details in dictionary.items():
    if condition:
        count += 1
```

18. Calculating Total Salary

To calculate the total salary of all employees:

```
total = 0

for employee, details in employees.items():
    total += details["salary"]

print(total)
```

For our example:

```
65000 + 50000 + 55000 + 60000
= 230000
```

19. Counting + Calculating Together

You can perform multiple calculations in one loop.

Example:

```
count = 0
it_salary = 0

for employee, details in employees.items():
    if details["department"] == "IT":
        count += 1
        it_salary += details["salary"]

print(f"Count: {count}")
print(f"IT Total Salary: {it_salary}")
```

This is efficient because you only loop through the dictionary once.

20. Important Patterns to Remember

Access

```
employees["emp1"]["salary"]
```

Update

```
employees["emp1"]["salary"] = 70000
```

Add inner key

```
employees["emp1"]["city"] = "Mumbai"
```

Add new employee

```
employees["emp4"] = {
    "name": "Neha",
    "department": "Marketing",
}
```

```
    "salary": 60000
}
```

Delete inner key

```
del employees["emp1"]["city"]
```

Delete complete employee

```
del employees["emp1"]
```

Loop

```
for employee, details in employees.items():
```

Access data inside loop

```
    details["name"]
    details["salary"]
    details["department"]
```

Filter

```
if details["salary"] >= 55000:
```

Store complete nested record

```
new_dict[employee] = details
```

21. Most Important Concept

Always identify **which level** you are working with.

```
employees
```

→ Outer dictionary

```
employees["emp1"]
```

→ Inner dictionary

```
employees["emp1"]["salary"]
```

→ Value inside the inner dictionary

Think:

```
employees  
  ↓  
  emp1  
    ↓  
    dictionary  
      ↓  
      salary  
        ↓  
        65000
```

Understanding these levels will make nested dictionaries much easier.